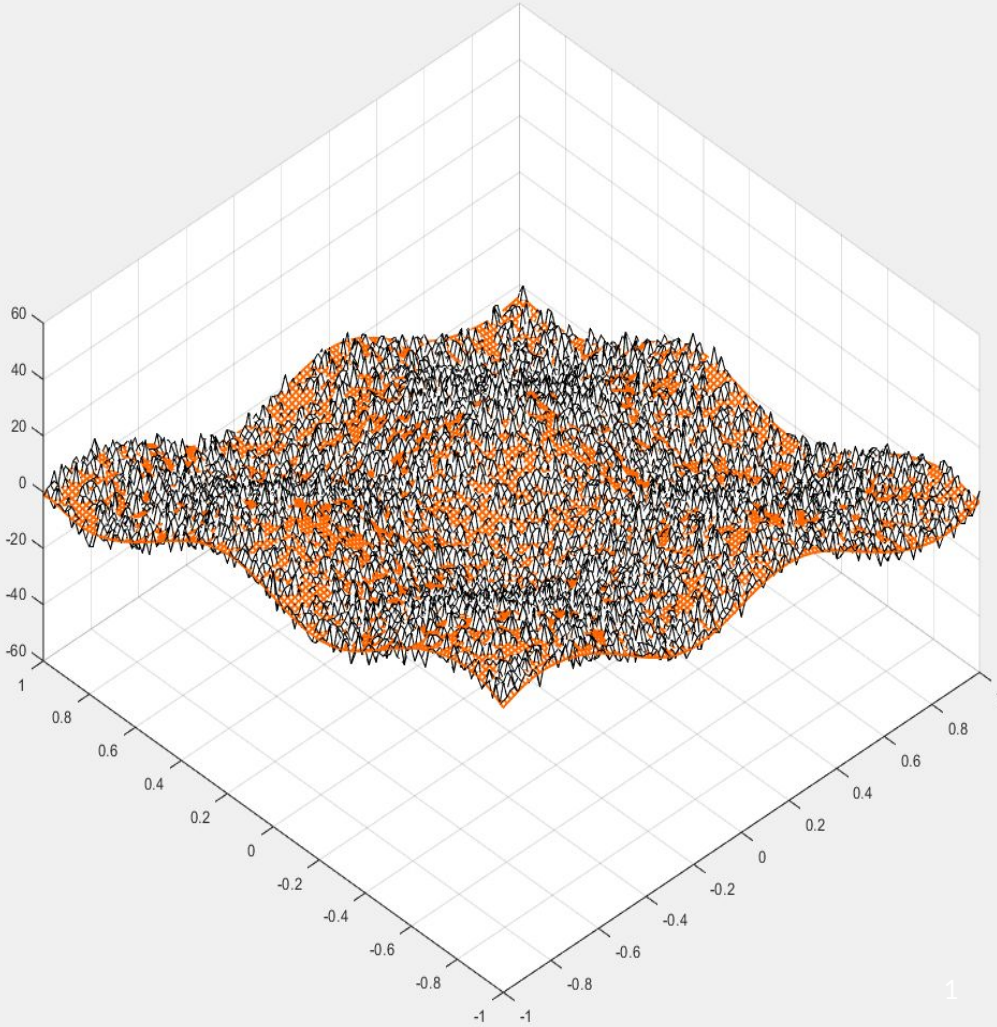
Aproximarea polinomială a unui sistem tridimensional

Moroșanu Ioan Teodor
Moroșan Sebastian-Vasile
Nagy David-Csaba

P.id: 8/7



1. Problemă vs Soluție

- **Probleme:**
  - Simularea fenomenelor fizice, complexitatea funcțiilor matematice, generalizarea modelului pentru date noi.

- **Soluție:**
  - Aproximare prin regresie polinomială, minimizarea erorii de aproximare, prevenirea fenomenelor de subantrenare și supraantrenare.

2. Descrierea soluției

1) <u>Structura aproximatorului:</u>

- Model polinomial cu **2 variabile.**

- Ecuația simplificată: $Y(x_1, x_2) = \sum_{i+j+k=n} 1^i \cdot x_1^j \cdot x_2^k$

- Regresori: intrările sunt transformate într-un **vector de regresori** (coloanele matricei Φ) ce reprezintă toate combinațiile de puteri până la un grad *n*.

2. Descrierea soluției

2) <u>Procedura de găsire a parametrilor θ</u>:

- Se rezolvă sistemul: $\Phi_{id} * \theta = Y_{id}$

- Se folosește metoda **erorii minime pătratice** pentru a obține cea mai optimă aproximare a sistemului.

3. Particularități

- Normalizarea datelor de intrare: din domeniul original -> intervalul **[-1, 1]**

- Alegerea unui grad suficient de mare pentru a se putea identifica pe grafic fenomenele de **subantrenare** și **supraantrenare.**

- Reconstrucția funcției pentru cea mai **mică eroare pătratică** obținută pe setul de date de validare.

3. Particularități

1) Normalizarea datelor de intrare:

   - Trecerea valorilor din domeniul original în intervalul **[-1, 1]** pentru a rezolva mai rapid **operațiile de ridicare la putere**.

   - După un grad specific, fără a normaliza datele, calculul **parametrilor** nu mai este **fezabil**.

3. Particularități

2) Identificarea fenomelor specifice:

- **Subantrenarea**: alegerea unui grad mult prea redus, ceea ce duce la o aproximare ineficientă.

- **Supraantrenarea**: alegerea unui grad mult prea mare, ceea ce duce la o aproximare ce percepe zgomotul.

3. Particularități

3) Construcția celui mai optim model:

- Identificarea celei mai **reduse erori** și construcția modelului specific acestuia.

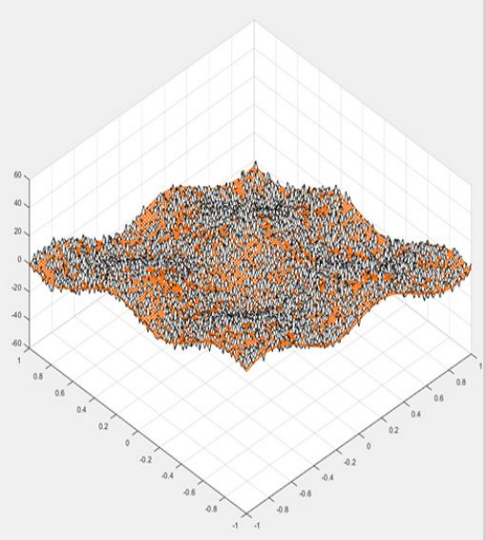
- Această particularitate este cea mai importantă deoarece reprezintă **aproximarea optimă** din punct de vedere al erorii.

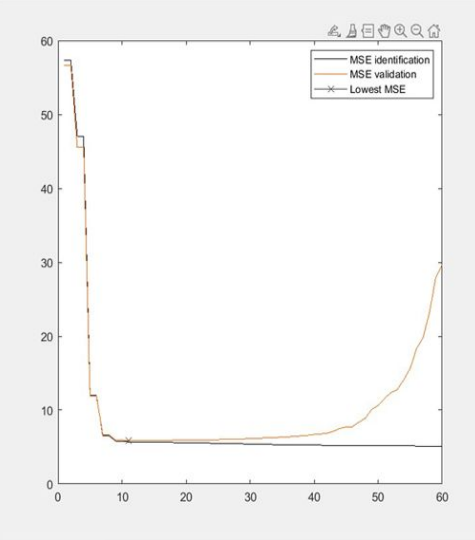
4. Rezultate

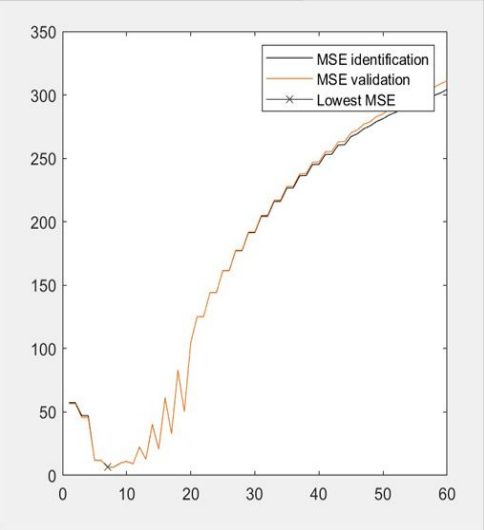
- Modelul obținut în urma celei mai **optime aproximări și gradul relevant.**

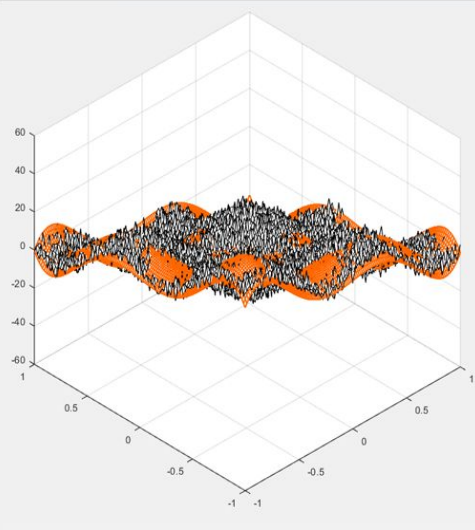
- **Erorile minime pătratice** în funcție de normalizarea datelor de intrare.

- Observații relevante (**fenomene**).

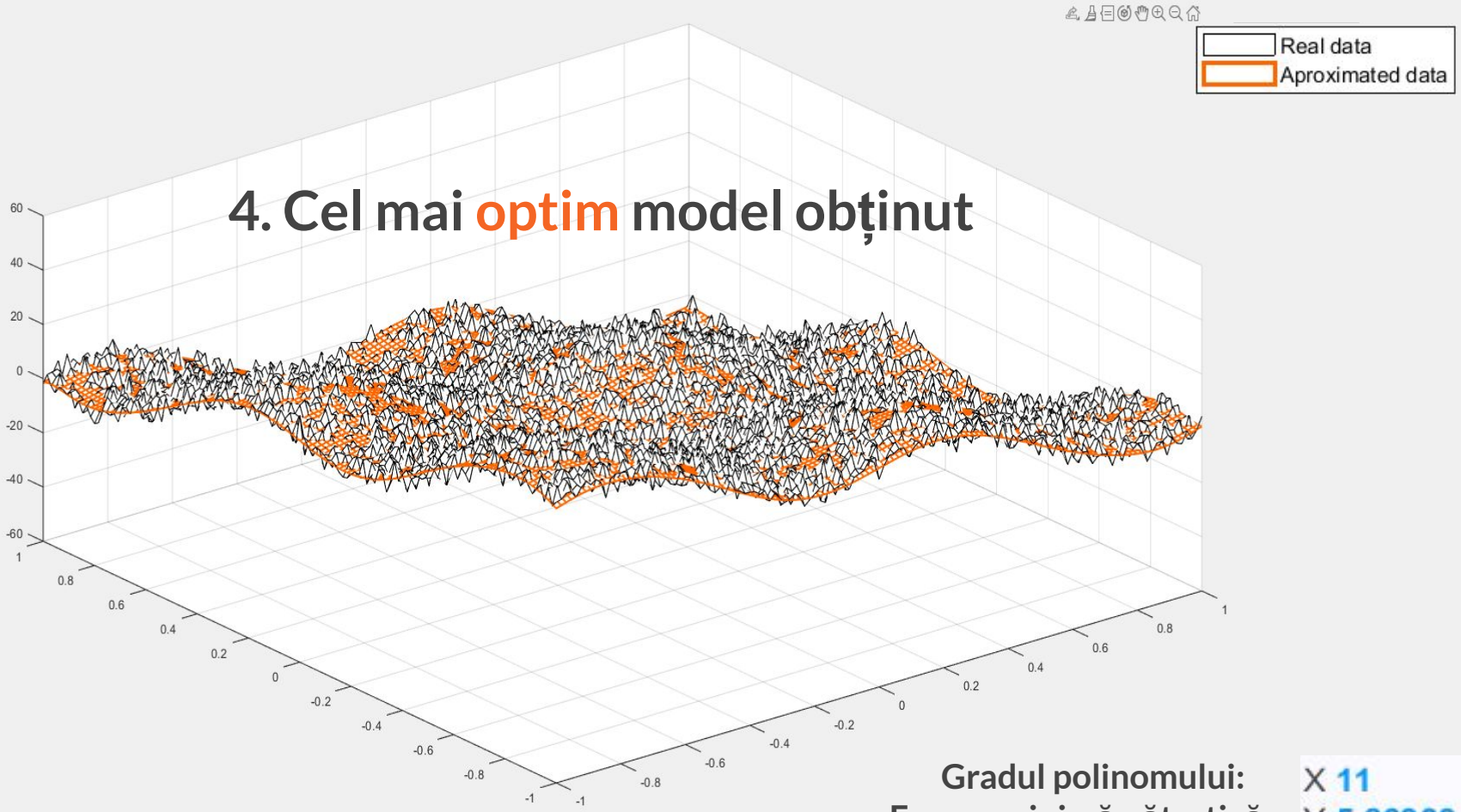








4. Cel mai optim model obținut



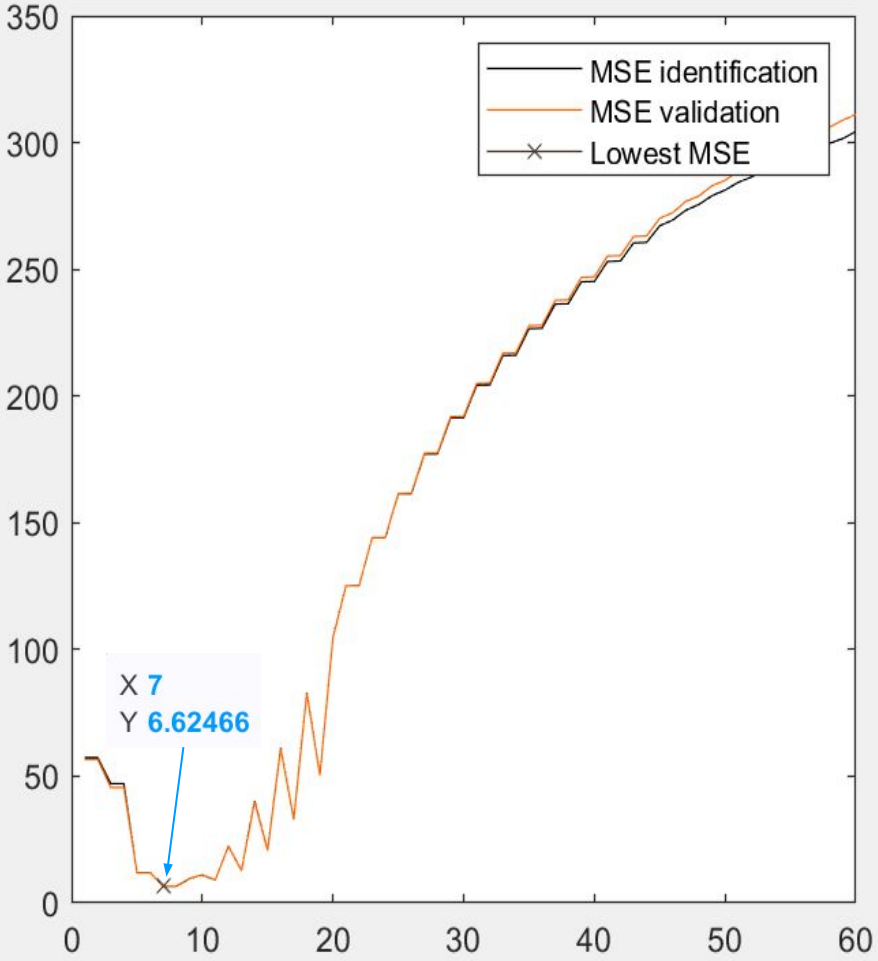
Gradul polinomului: X 11

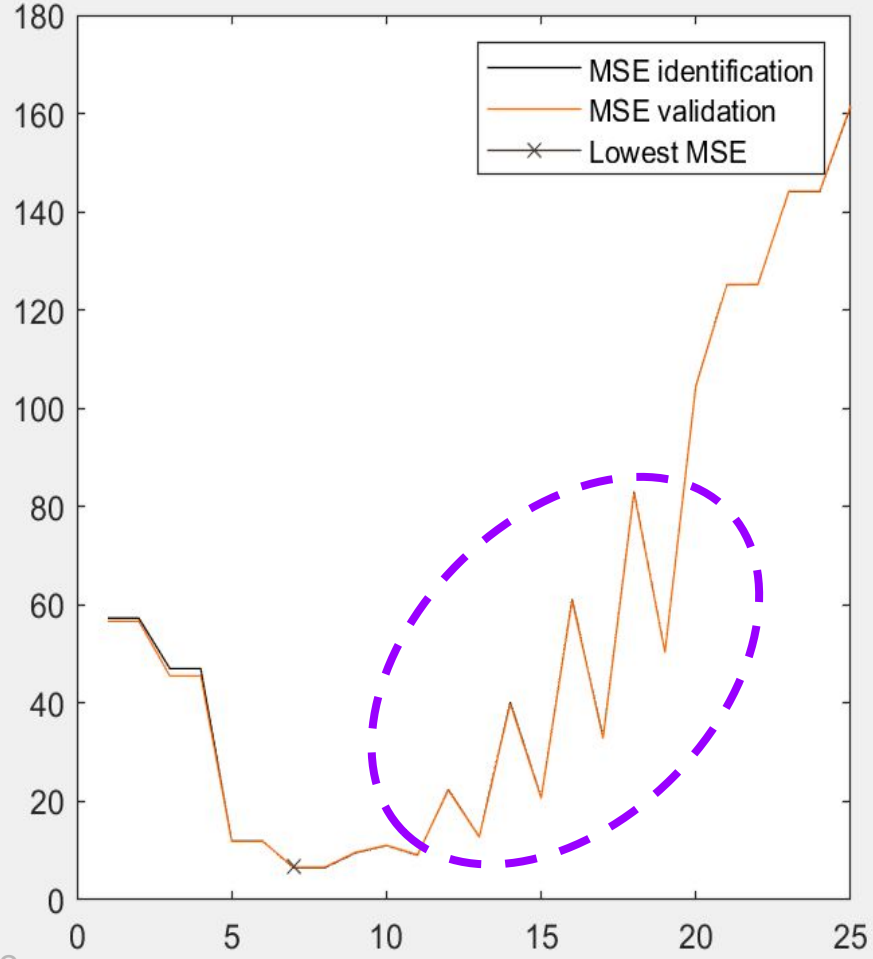
Eroare minimă pătratică: Y 5.86939

Eroarea minimă pătratică (date originale)

- Observăm că eroarea minimă pătratică pentru datele de identificare și validare sunt **aproape identice.**

- Eroarea cea mai redusă se poate identifica pentru **gradul 7** având o valoare de **6.62466.**





Observații (date originale)

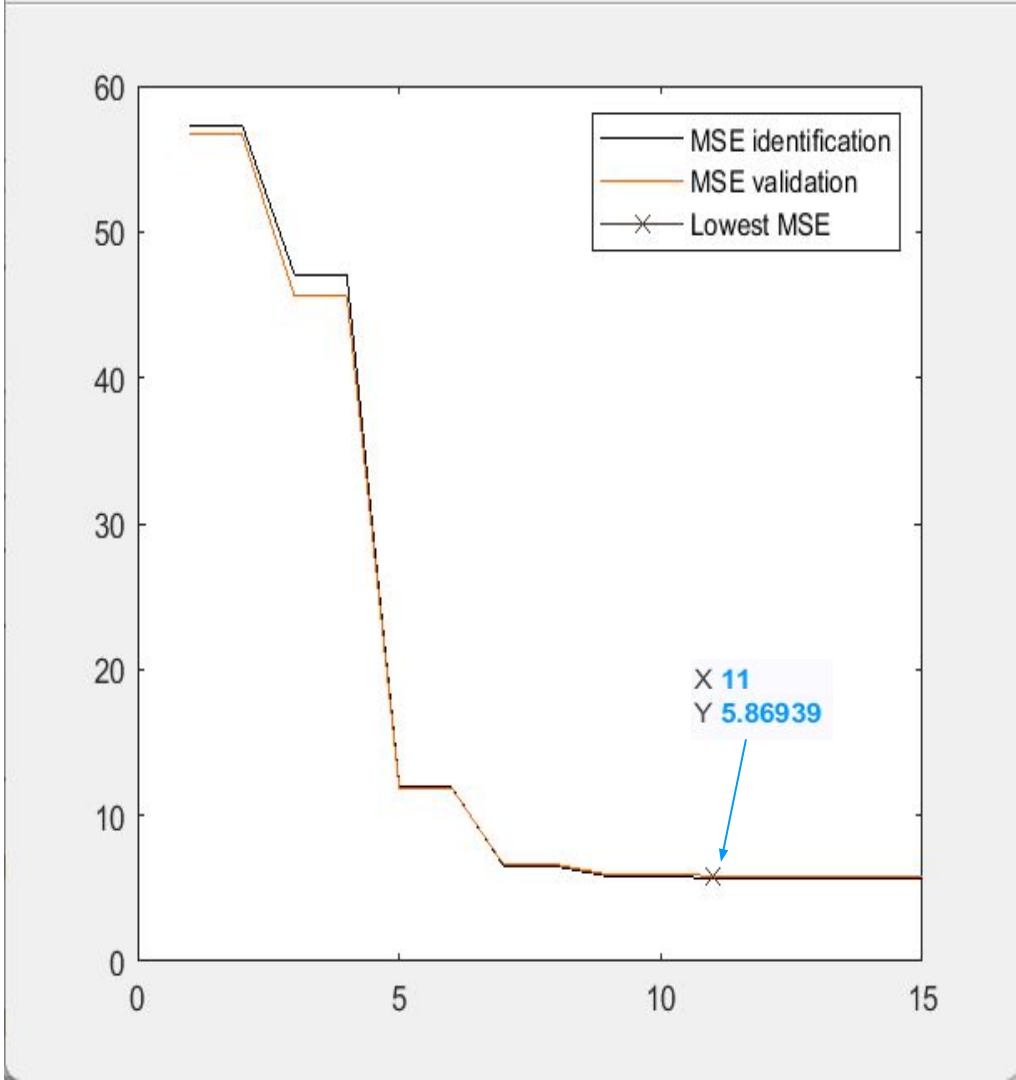
- Faptul că erorile minime pătratice pentru datele de identificare și cele de validare **coincid** reprezintă un comportament neașteptat.

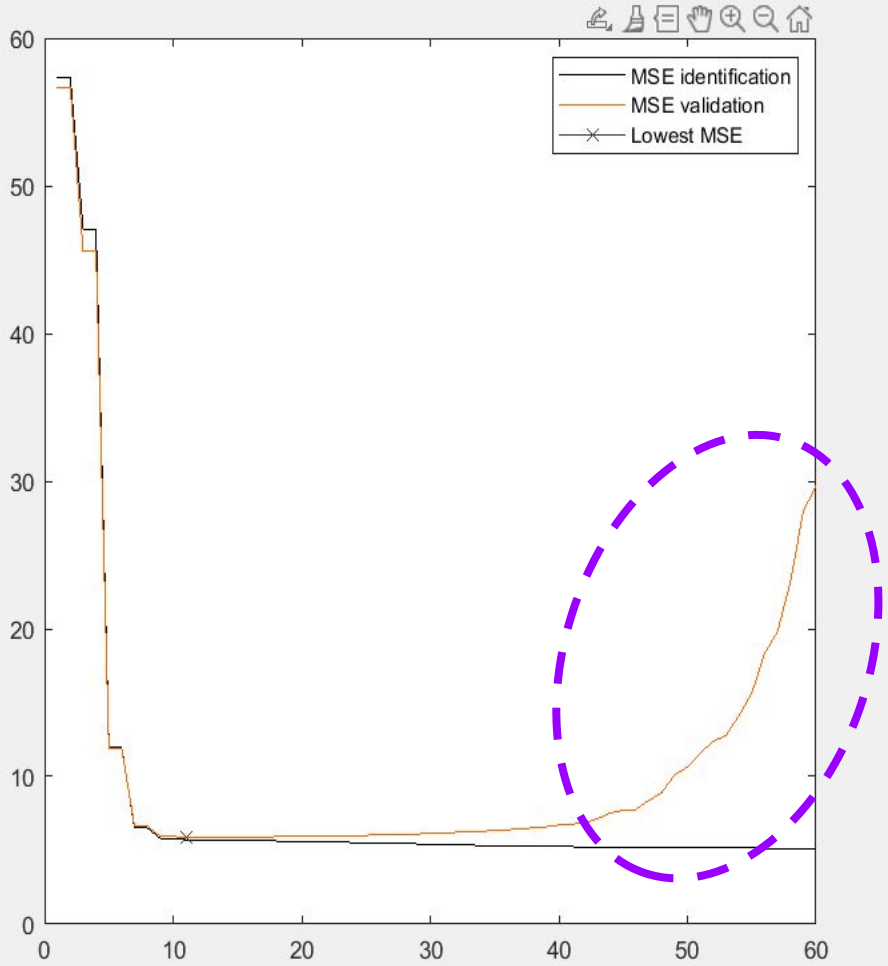
- Acest comportament apare deoarece intervin **instabilități numerice** specifice operațiilor cu valori de ordin mare.

EMP (date normalizate)

- Observăm că modelul aproximează din ce în ce mai optim, **proporțional** cu creșterea gradului.

- Eroarea cea mai redusă se poate identifica pentru **gradul 11** având o valoare de **5.86939.**





Observații

- Pentru **grade suficient de mici** (1-6) apare fenomenul de **subantrenare**. Acest comportament implică o eroare substanțială, aproximarea fiind una ineficientă.

- În cealaltă extremă, pentru grade mari, modelul începe să perceapă **zgomotele**. Acest comportament reprezintă fenomenul de **supraantrenare.**

5. Concluzii

Scopul proiectului constă în aproximarea polinomială a unui sistem tridimensional prin metoda de regresie, punând în evidență limitele modelării polinomiale, cât și efectul normalizării datelor și fenomenele specifice identificării sistemelor (subantrenare și supraantrenare).

6. Anexă: cod sursă - normalizarea datelor

```
idInput1Size = length(id.X{1});
idInput2Size = length(id.X{2});
valInput1Size = length(val.X{1});
valInput2Size = length(val.X{2});

idInput1_min = min(id.X{1});  idInput1_max = max(id.X{1});
idInput2_min = min(id.X{2});  idInput2_max = max(id.X{2});

idInput1 = 2 * (id.X{1} - idInput1_min)/(idInput1_max - idInput1_min) - 1;
idInput2 = 2 * (id.X{2} - idInput2_min)/(idInput2_max - idInput2_min) - 1;
valInput1 = 2 * (val.X{1} - idInput1_min)/(idInput1_max - idInput1_min) - 1;
valInput2 = 2 * (val.X{2} - idInput2_min)/(idInput2_max - idInput2_min) - 1;

idOutput = id.Y;
valOutput = val.Y;
```

6. Anexă: cod sursă - generarea puterilor regresorilor

```
maxPolynomGrade = 20;
idMSE = zeros(maxPolynomGrade, 1);
valMSE = zeros(maxPolynomGrade, 1);
minMSE = 1000000;

for polynomGrade = 1 : maxPolynomGrade
    regNumber = (polynomGrade + 1) * (polynomGrade + 2) / 2;
    regPowers = zeros(regNumber, 3);
    i = 1;
    for power1 = polynomGrade : -1 : 0
        for powerX1 = polynomGrade - power1 : -1 : 0
            powerX2 = polynomGrade - power1 - powerX1;
            regPowers(i, 1) = power1;
            regPowers(i, 2) = powerX1;
            regPowers(i, 3) = powerX2;
            i = i + 1;
        end
    end
```

6. Anexă: cod sursă - generarea matricelor Φ

```
idInputsSize = idInput1Size * idInput2Size;
phiId = zeros(idInputsSize, regNumber);
inputIndex1 = 1;
inputIndex2 = 1;
for i = 1 : idInputsSize
    for j = 1 : regNumber
        phiId(i, j) = 1^regPowers(j, 1) * idInput1(inputIndex1)^regPowers(j, 2) * idInput2(inputIndex2)^regPowers(j, 3);
    end
    inputIndex1 = inputIndex1 + 1;
    if mod(i, idInput1Size) == 0
        inputIndex2 = inputIndex2 + 1;
        inputIndex1 = 1;
    end
end

valInputsSize = valInput1Size * valInput2Size;
phiVal = zeros(valInputsSize, regNumber);
inputIndex1 = 1;
inputIndex2 = 1;
for i = 1 : valInputsSize
    for j = 1 : regNumber
        phiVal(i, j) = 1^regPowers(j, 1) * valInput1(inputIndex1)^regPowers(j, 2) * valInput2(inputIndex2)^regPowers(j, 3);
    end
    inputIndex1 = inputIndex1 + 1;
    if mod(i, valInput1Size) == 0
        inputIndex2 = inputIndex2 + 1;
        inputIndex1 = 1;
    end
end
```

6. Anexă: cod sursă - identificarea parametrilor θ

```
outputIndex1 = 1;
outputIndex2 = 1;
idOutputVector = zeros(idInputsSize, 1);
for i = 1 : idInputsSize
    idOutputVector(i) = idOutput(outputIndex1, outputIndex2);
    outputIndex1 = outputIndex1 + 1;
    if mod(i, idInput1Size) == 0
        outputIndex2 = outputIndex2 + 1;
        outputIndex1 = 1;
    end
end

teta = phiId \ idOutputVector;
outputIndex1 = 1;
outputIndex2 = 1;
valOutputVector = zeros(valInputsSize, 1);
for i = 1 : valInputsSize
    valOutputVector(i) = valOutput(outputIndex1, outputIndex2);
    outputIndex1 = outputIndex1 + 1;
    if mod(i, valInput1Size) == 0
        outputIndex2 = outputIndex2 + 1;
        outputIndex1 = 1;
    end
end

idPredictionOutputVector = phiId * teta;
valPredictionOutputVector = phiVal * teta;
idMSE(polynomGrade) = (1/idInputsSize) * sum((idOutputVector - idPredictionOutputVector).^2);
valMSE(polynomGrade) = (1/valInputsSize) * sum((valOutputVector - valPredictionOutputVector).^2);
```

6. Anexă: cod sursă - afișarea rezultatelor

```
valPredictionOutput = zeros(valInput1Size, valInput2Size);
vectorIndex = 1;
for i = 1 : valInput1Size
    for j = 1 : valInput2Size
        valPredictionOutput(i, j) = valBestPredictionOutputVector(vectorIndex);
        vectorIndex = vectorIndex + 1;
    end
end

figure;
mesh(valInput1, valInput2, valOutput, 'EdgeColor', 'k', 'LineWidth', 0.5);
hold on;
mesh(valInput1, valInput2, valPredictionOutput, 'EdgeColor', [1, 0.4, 0], 'LineWidth', 1.5);
legend("Real data", "Aproximated data");
range = 1 : maxPolynomGrade;
figure;
plot(range, idMSE, 'Color', 'k');
hold on
plot(range, valMSE, 'Color', [1, 0.4, 0]);
hold on;
plot(MSEgrade, minMSE, 'Color', [0.2745, 0.2235, 0.2000], 'Marker', 'x');
legend("MSE identification", "MSE validation", "Lowest MSE");
```